Support Triage Assistant — Collected Learnings

A running log of what was built and, more importantly, what was learned while building a support-ticket triage system with the Anthropic API — and then folding it into the ninjamountain monorepo and putting it under a real evaluation harness.

The original day-by-day notes (Days 1–2) live in `experiments/LESSONS.md`; this document collects them with everything that came after.

---

1. The project at a glance

The triage pipeline takes a support ticket (`subject` + `body`) and returns a structured judgment from Claude:

- **category** — one of bug, config, billing, how_to, feature_request
- **severity** — one of low, medium, high, critical
- **summary** — one sentence
- **suggested_first_response** — a draft reply
- **needs_engineering_escalation** — boolean

It never throws: every call resolves to a typed `TriageOutcome` (success with a result, or failure with a reason). It retries transient API errors with backoff, and re-prompts the model with a corrective hint when output fails to parse or validate.

Architecture (single responsibility per file):

- `client.ts` — the Anthropic API call (typed errors, exponential backoff)
- `parse.ts` — extract JSON from raw model output
- `validate.ts` — validate output against the contract (returns errors, never throws)
- `triage.ts` — orchestration: the retry + correction loop
- `types.ts` — domain and return types

---

2. Days 1–2 — Foundation (summary)

Day 1 (JavaScript). Built the core pipeline with a two-layer retry strategy: API failures retried in `client.js` with backoff; output failures retried in `triage.js` with a corrective hint appended to the next prompt. The validator returns `{ valid, errors }` instead of throwing, keeping the orchestration layer in control.

Key bug caught in review: a typo in the system prompt (`suggested_fist_response`) meant every call failed validation on the first attempt and burned a retry — and the type system could not catch it because the field name lived inside a string literal.

Day 2 (TypeScript port). Same logic, but types replaced documentation: a discriminated union (`TriageOutcome`) the compiler enforces at every call site, `unknown` for unvalidated model output, and typed SDK errors (`instanceof APIError`) instead of status guessing. The lasting lesson: **static types stop at the string boundary.** The prompt and the `TriageResult` interface can drift without the compiler noticing — which is exactly why runtime validation stays essential.

---

3. Day 3 — Labeling the gold dataset

Day 3 was supposed to be one task: make sure the labels accurately reflect the tickets. It turned out to be the foundation for everything that follows.

The labels are the asset, not the code. A scoring script is easy; a trustworthy set of gold labels is the hard, valuable part. The initial `labels.json` was placeholder data (18 of 19 entries identical), which would have produced meaningless — even deceptively good — scores. Garbage labels in, garbage metrics out.

Lessons:

- Read the full ticket body before labeling — the subject is not enough.
- Severity is far more subjective than category; calibrate it deliberately (a billing question is low; an account suspension is critical).
- Hand-labeling 20 tickets surfaces the genuinely ambiguous cases — and those ambiguities are signal, not noise. They tell you where your definitions are underspecified.

---

4. Day 4 — Evaluation-driven iteration

This was the heart of the project: building a scored accuracy report and using it to drive changes. The metric journey:

Stage	Category	Severity exact	Severity MAE	Trustworthy?
First baseline (temp 1)	90%	55%	0.50	No — noisy
Severity rubric v1 (temp 1)	90%	50%	0.55	No — noisy
Rubric v2 + temperature 0	85%	45%	0.60	Yes — stable
Reconciled labels to one rule	90%	85%	0.15	Yes
"No workaround = high" policy	85%	90%	0.10	Yes
Category rubric + T18 relabel	90%	85%	0.15	Yes

The numbers going *down* at the temperature-0 step was the most important moment: it revealed that the earlier higher scores were partly luck, not skill.

The lessons, in order of how hard they hit:

1. **Vibes do not scale — measure.** "The triage feels good" becomes "category is 90%, severity is stuck at 50%, and here is exactly where it fails."
2. **Pin `temperature: 0` for evaluation.** At the default temperature the model rolls dice; run-to-run variance on 20 tickets is several points. You cannot measure a prompt change while the ground moves under you. A lower-but-stable score beats a higher-but-random one.
3. **A failing eval often means the labels are wrong, not the model.** Two tickets (T02 a suspected-fraud billing case, T18 a platform ignoring a correct setting) were gold-label errors the score exposed. The model was right; the labels were not.
4. **Make the specification self-consistent.** Most of the severity jump (45% to 85%) came from reconciling the labels to one written rule — not from the model improving. Being honest about *why* a number moved matters as much as the number.
5. **A shared prompt couples tasks.** Editing the severity wording shifted category predictions; adding a category rubric nudged a severity label. Always score the *whole* report after any change, not just the metric you were targeting.
6. **Written policy beats per-case instinct.** A severity rubric is operational policy. Its value is not that it is right on every ticket — it is that it is *decided*, which buys consistency, speed, and defensibility. "No workaround = high" is a clean, teachable line that ends the per-ticket debate.
7. **There is a small-data ceiling.** Past a point, every prompt edit just trades one borderline ticket for another, and the metric swings because the sample is tiny. The next lever is a bigger dataset, not more prompt clauses.

Where it landed: category 90% (config recall 1.0), severity 90% exact / 100% off-by-one / MAE 0.10. The remaining misses are genuine model weaknesses (bug reports phrased as polite questions get read as how_to) — the honest, interesting kind of leftover.

---

5. Engineering & workflow lessons

Folding the standalone experiment into ninjamountain taught a parallel set of lessons that had nothing to do with the model:

- **One source of truth.** The triage logic became a workspace package (`@ninjamountain/triage`) consumed by the web app via `transpilePackages`, so there is no duplicated copy to drift. The Python/JS ports were kept as frozen reference snapshots under `experiments/`, explicitly not wired into the build.
- **Preserve before you retire.** The old standalone repo's only copy of the TS+Python work was an unpushed commit. Push first, then archive, then delete — never destroy the only copy.
- `.env` **is not auto-trusted.** A subtle one: `dotenv` will not overwrite an already-defined environment variable, so an empty `ANTHROPIC_API_KEY` in the environment silently beat the value in `.env`. Loading with `override: true` (and an explicit path) fixed it. Also: Next.js `.env.local` changes require a dev-server restart.
- **Keep secrets out of git.** API keys live only in gitignored `.env` / `.env.local` files; a pre-publish scan confirmed nothing leaked before the repo went public.

- **A real test loop is worth building.** Two harnesses — the web UI (`/projects/triage`) and a CLI (`npm run triage`) — make iterating far faster than a hardcoded script.

---

6. What is this field called?

The evaluation work has a name — several, depending on how academic you want to be.

The umbrella term is model evaluation, or simply "evals." Applied to large language models it is often called **LLM evaluation**, and the practice of building a metric and letting it drive changes is **evaluation-driven development** (the ML cousin of test-driven development).

It is not one field but a convergence of several:

- **Statistical classification / pattern recognition** — the source of accuracy, precision, recall, F1, and the confusion matrix. Our category scoring is textbook multi-class classification evaluation.
- **Information retrieval (IR)** — where precision and recall originated (how many relevant documents did a search return, and how many of the returned ones were relevant).
- **Psychometrics & measurement theory** — the science of measuring fuzzy human judgments reliably. This is where label quality, *inter-annotator agreement*, and *reliability vs validity* come from. Our whole Day-3/4 struggle — getting labels consistent enough to trust — is a psychometrics problem.
- **Statistics & experimental design** — sampling, variance, significance, and A/B testing. The "is this change real or noise?" question is a statistics question, and the small-data ceiling is about sample size.
- **Ordinal data analysis** — severity is an *ordinal* variable (low < medium < high < critical), so an off-by-one miss is not as bad as off-by-three. That is why we tracked mean absolute error, not just exact match.

For this specific project, the precise label is **supervised text-classification evaluation against a gold-labeled dataset** — with one ordinal target (severity) and one nominal target (category).

---

7. Glossary

Term	Meaning
Ground truth / gold label	The human-assigned correct answer a prediction is scored against.
Golden dataset	The curated set of inputs plus gold labels used for evaluation.
Accuracy	Fraction of predictions that exactly match the gold label.
Precision	Of the items predicted to be class X, how many truly were X.
Recall	Of the items that truly are class X, how many were caught.
F1	Harmonic mean of precision and recall — one number balancing both.
Support	How many gold examples exist for a class (the denominator behind recall).
Confusion matrix	A grid of "predicted X but actually Y," exposing systematic mistakes.

Term	Meaning
Nominal variable	A category with no order (bug vs billing).
Ordinal variable	A category with a meaningful order (low < ... < critical).
Mean absolute error (MAE)	Average distance between predicted and true rank — for ordinal targets.
Off-by-one	An ordinal prediction one step from the truth (high vs critical).
Inter-annotator agreement	How consistently independent humans assign the same labels; a measure of label reliability.
Temperature	Sampling randomness; 0 makes the model (near-)deterministic.
Baseline	The reference score a change is measured against.
Regression	A change that makes a previously-good metric worse.
Label noise	Errors or inconsistencies in the gold labels themselves.
Class imbalance	Some classes having far more examples than others, skewing metrics.
Eval harness	The script/tooling that runs predictions and computes the report.
Evaluation-driven development	Building a metric first, then iterating against it.

---

8. Command reference

Command	What it does
`npm --workspace @ninjamountain/triage run triage`	Triage a built-in sample ticket.
`npm ... run triage -- --id T02`	Triage one dataset ticket by id.
`npm ... run triage -- --all`	Triage every ticket in the dataset.
`npm ... run score`	Full scored accuracy report vs gold labels.
`npm ... run typecheck`	Type-check the package.
`npm run dev:web`	Start the Next.js app; UI at /projects/triage.

---

9. Where to go next

- **Grow the dataset** to 40–60 tickets — the single highest-leverage move, since it stops the metrics from swinging on one ticket.
- **Advance the curriculum** — RAG (retrieve similar past tickets) and tool-use, now that there is a regression harness to prove they actually help.

- **One surgical prompt fix** for the question-framed-bug confusion — but verify it does not ripple into other tickets.